

M4: Creating Responsive and Accessible UIs for Android Apps

June 2, 2026 3:54 PM

Overview

Introduction

Hello and welcome to Module 4 of COMP-304. In this module, you will learn about creating Responsive and Accessible UIs for Android Apps.

Learning Outcomes

At the end of this module, you will be able to:

- 1. Discuss the fundamental principles and features of Material Design 3.
- 2. Identify and utilize various Material Design components to enhance user interfaces.
- 3. Apply Material Design 3 color schemes to create visually appealing and consistent app themes.
- 4. Design responsive UIs that adapt to large screens and foldable devices using WindowSize Class.
- 5. Implement accessibility features to ensure apps are usable by all users, including those with disabilities.

Checklist

There are several tasks you'll need to complete:

- 1. Review the online content for this module, which includes reviewing the module's key terms and concepts.
- 2. Run all class examples.

Key Terms and Concepts

Here are a few terms you need to be familiar with.

Term	Definition
Material Design 3	Google's latest design system for creating aesthetically pleasing and functional UIs for Android apps.
Dynamic Color System	Automatically adjusts app colors to match the user's wallpaper, ensuring a personalized appearance.
Top App Bars	Key components for navigation and information display, with four types: small, medium, large, and center-aligned.
FloatingActionButton (FAB)	Represents the primary action in an application, typically positioned at the bottom right of the screen.
Bottom App Bars	Display navigation items and actions at the bottom of the screen, ideal for primary destinations within the app
Scaffold Structure	A fundamental layout structure in Material Design 3 that supports top bar, bottom bar, and FAB, simplifying the layout process.
Color Schemes	A tool for generating and exporting theme settings for an app, allowing customization of primary, secondary, and tertiary colors.
Material Theme Builder	Use this method to instantiate UI objects in Kotlin code
WindowSizeClass	A strategy for designing responsive UIs that adapt to varying screen sizes and orientations, especially for large screens and foldable devices.
Accessibility	Ensuring apps are usable by all users, including those with disabilities, through semantics, content descriptions, and proper sizing.
@OptIn(ExperimentalMaterial3Api::class)	Annotation used to indicate the use of experimental APIs in

	sizing.
@OptIn(ExperimentalMaterial3Api::class)	Annotation used to indicate the use of experimental APIs in Material Design 3
Typography	Simplified and clear typography guidelines provided by Material Design 3 for improved readability
Responsive Design	Designing UIs that adapt seamlessly to different screen sizes and orientations using Material Design 3 guidelines
Component Library	A collection of UI components provided by Material Design 3, including app bars, buttons, cards, and more.

Term	Definition
Material Design 3	Google's latest design system for creating aesthetically pleasing and functional UIs for Android apps.
Dynamic Color System	Automatically adjusts app colors to match the user's wallpaper, ensuring a personalized appearance.
Top App Bars	Key components for navigation and information display, with four types: small, medium, large, and center-aligned.
FloatingActionButton (FAB)	Represents the primary action in an application, typically positioned at the bottom right of the screen.
Bottom App Bars	Display navigation items and actions at the bottom of the screen, ideal for primary destinations within the app
Scaffold Structure	A fundamental layout structure in Material Design 3 that supports top bar, bottom bar, and FAB, simplifying the layout process.
Color Schemes	A tool for generating and exporting theme settings for an app, allowing customization of primary, secondary, and tertiary colors.
Material Theme Builder	Use this method to instantiate UI objects in Kotlin code
WindowSizeClass	A strategy for designing responsive UIs that adapt to varying screen sizes and orientations, especially for large screens and foldable devices.
Accessibility	Ensuring apps are usable by all users, including those with disabilities, through semantics, content descriptions, and proper sizing.
@OptIn(ExperimentalMaterial3Api::class)	Annotation used to indicate the use of experimental APIs in Material Design 3
Typography	Simplified and clear typography guidelines provided by Material Design 3 for improved readability
Responsive Design	Designing UIs that adapt seamlessly to different screen sizes and orientations using Material Design 3 guidelines
Component Library	A collection of UI components provided by Material Design 3, including app bars, buttons, cards, and more.

From <https://luminare.centennialcollege.ca/ultra/courses/_39572_1/outline/edit/document/_5520527_1?courseId=_39572_1&view=content&state=view>

Weekly Topic

The topic this week is about Creating Responsive and Accessible UIs for Android Apps.

The following resources provide great insights into Material Design 3 in Compose:

1. <https://developer.android.com/develop/ui/compose/designsystems/material3>

COMP-304: Mobile apps Development

Summary

Key Take-Aways

Here are a few key take-aways from this module.

- Material Design 3 is Google's latest design system.
- Helps in **creating aesthetically pleasing and functional UIs** for Android apps.
- Provides **comprehensive guidelines and pre-built components**

- Material Design 3 is Google's latest design system.
- Helps in **creating aesthetically pleasing and functional UIs** for Android apps.
- Provides **comprehensive guidelines and pre-built components**.
- Focuses on improving **visual aesthetics and usability**.
- **Simplifies integration** with Android development environments.

Don't forget to complete the items on your checklist.

Summary Checklist

Don't forget to complete the items on your checklist:

1. Review the online content for this module, which includes reviewing the module's key terms and concepts.
2. Read articles/tutorials given in Learning Activities.
3. Keep working on Lab Assignment 2

- Works Cited
- <https://developer.android.com/develop/ui/views/theming/look-and-feel>

Coming up next, we'll be looking at weekly topic for module 5 as identified in the approved course outline.

COMP-304: Mobile Apps Development

<https://developer.android.com/develop/ui/views/theming/look-and-feel>:

Android Developers > Develop > Core areas > UI > Views

Was this helpful?  

Material Design for Android

Try the Compose way

Jetpack Compose is the recommended UI toolkit for Android. Learn how to work with theming in Compose.

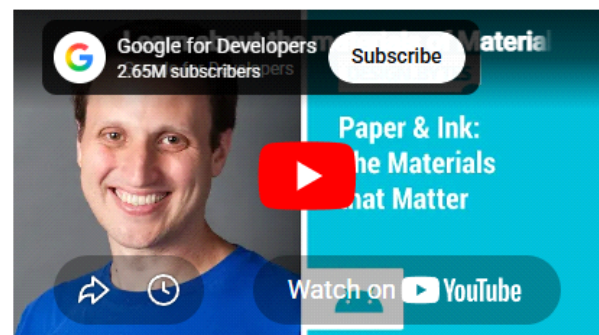
[Material Design 3](#) →



Material Design is a comprehensive guide for visual, motion, and interaction design across platforms and devices. To use Material Design in your Android apps, follow the guidelines defined in the [Material Design specification](#). If your app uses Jetpack Compose, you can use the [Compose Material 3](#) library. If your app uses views, you can use the [Android Material Components](#)  library.

Android provides the following features to help you build Material Design apps:

- A Material Design app theme to style all your UI widgets
- Widgets for complex views, such as lists and cards
- APIs for custom shadows and animations



YouTube video player

Material theme and widgets

To take advantage of the Material features, such as styling for standard UI widgets, and to streamline your app's style definition, apply a Material-based theme to your app.



definition, apply a Material-based theme to your app.

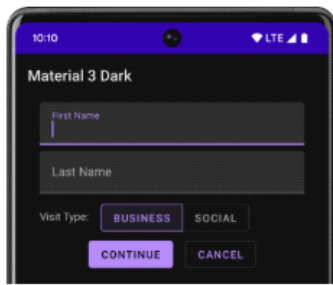


Figure 1. Dark Material theme.

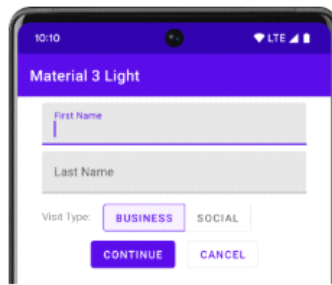


Figure 2. Light Material theme.

If you use Android Studio to create your Android project, it applies a Material theme by default. To learn how to update your project's theme, see [Styles and themes](#).

To provide your users a familiar experience, use Material's most common UX patterns:

- Promote your UI's main action with a [floating action button](#) (FAB).
- Show your brand, navigation, search, and other actions using the [app bar](#).
- Show and hide your app's navigation with the [navigation drawer](#).
- Choose from the many other Material Components for your app layout and navigation, such as collapsing toolbars, tabs, a bottom nav bar, and more. To see them all, see the [Material Components for Android catalog](#).

Whenever possible, use predefined Material Icons. For example, for the navigation "menu" button for your navigation drawer, use the standard "hamburger" icon. See [Material Design Icons](#) for a list of available icons. You can also import SVG icons from the Material Icon library with Android Studio's [Vector Asset Studio](#).

Elevation shadows and cards

In addition to the *X* and *Y* properties, views in Android have a *Z* property. This property represents the elevation of a view, which determines the following:

- The size of its shadow: views with higher *Z* values cast bigger shadows.
- The drawing order: views with higher *Z* values appear on top of other views.

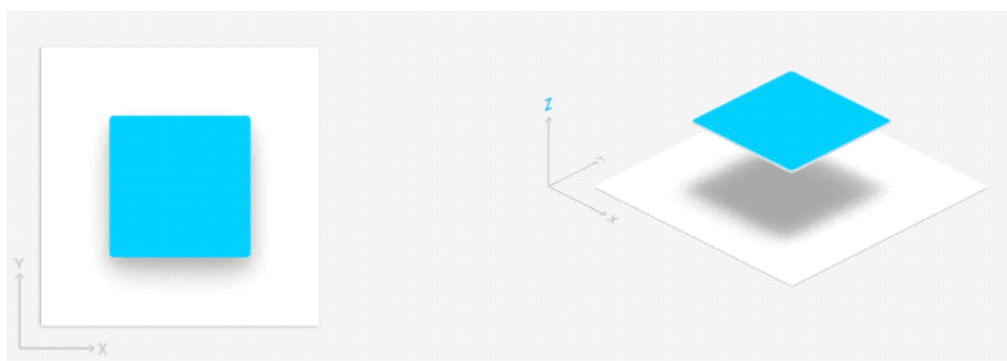


Figure 3. The *Z* value representing elevation.

You can apply elevation to a card-based layout, which helps you display important pieces of information inside cards that provide a Material look. You can use the `CardView` widget to create cards with a default elevation. For more information, see [Create a card-based layout](#).

For information about adding elevation to other views, see [Create shadows and clip views](#).

Animations

Animation APIs let you create custom animations for touch feedback in

Animations

Animation APIs let you create custom animations for touch feedback in UI controls, changes in view state, and activity transitions.

These APIs let you:

- Respond to touch events in your views with **touch feedback** animations.
- Hide and show views with **circular reveal** animations.
- Switch between activities with custom **activity transition** animations.
- Create more natural animations with **curved motion**.
- Animate changes in one or more view properties with **view state change** animations.
- Show animations in **state list drawables** between view state changes.

Touch feedback animations are built into several standard views, such as buttons. The animation APIs let you customize these animations and add them to your custom views.

For more information, see [Introduction to animations](#).

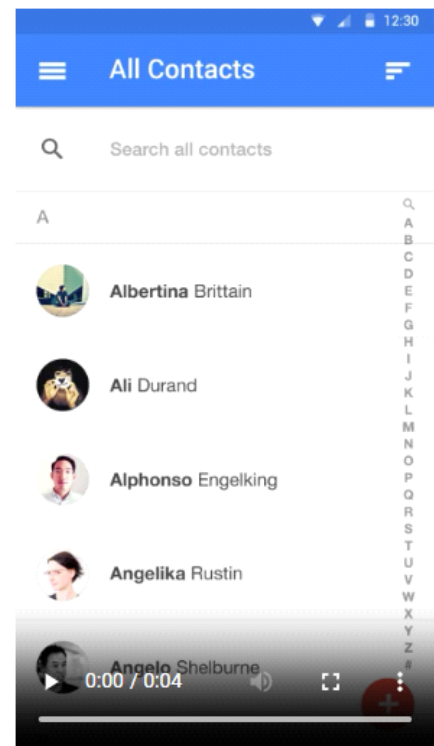


Figure 4. A touch feedback animation.

Drawables



TRY THE COMPOSE WAY

Work with Drawables in Compose



These capabilities for drawables help you implement Material Design apps:

- **Vector drawables** are scalable without losing definition and are perfect for single-color in-app icons. Learn more about [vector drawables](#).
- **Drawable tinting** lets you define bitmaps as an alpha mask and tint them with a color at runtime. See how to [add tint to drawables](#).
- **Color extraction** lets you automatically extract prominent colors from a bitmap image. See how to [select colors with the Palette API](#).